

CSCI 361 Lecture 11: Turing Machines Cont.

Shikha Singh

Announcements & Logistics

- **HW 5** was due last night, HW 6 will be released today (due next Wed)
- Hand in **Exercise # 8**, pick up **Exercise # 9**
- Reminder: Colloquium tomorrow 2.35 pm in Wege
 - What I did Last Summer (Research Edition)

Last Time

- Wrapped up CFGs
- Introduced Turing Machines and looked at some examples

Today

- Continue with discussion on variants of Turing machines
- Church-Turing Thesis and discussion of decidable languages
- Change in pace: we need a lot of setup for this half of the course
 - So may not get to "board exercises" until we finish this section
 - Soon, we will have plenty of reductions to do on the board

Last Time: Levels of Description

- Low-level description using δ and state diagram provides a complete picture but quickly become unwieldy
- Stick to "medium-level" description from now on
 - Describes how the TM works in English
 - What is OK: can include anything in a high-level description, as long as you are convinced that, if you had to, you could design a (low-level) Turing machine for it!
- We will move on to high-level descriptions (algorithms) later

Turing Machine Description

- **Exercise.** Give a medium-level description of a TM that recognizes $L = \{a^n b^n c^n \mid n \geq 0\}$

Exercise. Give a medium-level description of a TM that recognizes $L = \{a^n b^n c^n \mid n \geq 0\}$

Medium Level Description.

In the following we assume that tape symbols $x, y, z \notin \Sigma$

M = “On input string w :

1. Scan the input from left to right to check if it is of the form $a^*b^*c^*$ and reject if it is not.
2. Return head to left hand end of tape.
3. While there are a's remaining on the tape, do:
 - Replace the first a with an x, scan right until a b occurs; replace it with y, and scan to the right until a c occurs; replace it with z. If the corresponding b and c for each a are not found, reject.
4. If there are no b's or c's remaining on the tape, accept. Otherwise, reject.

Many Models of TM

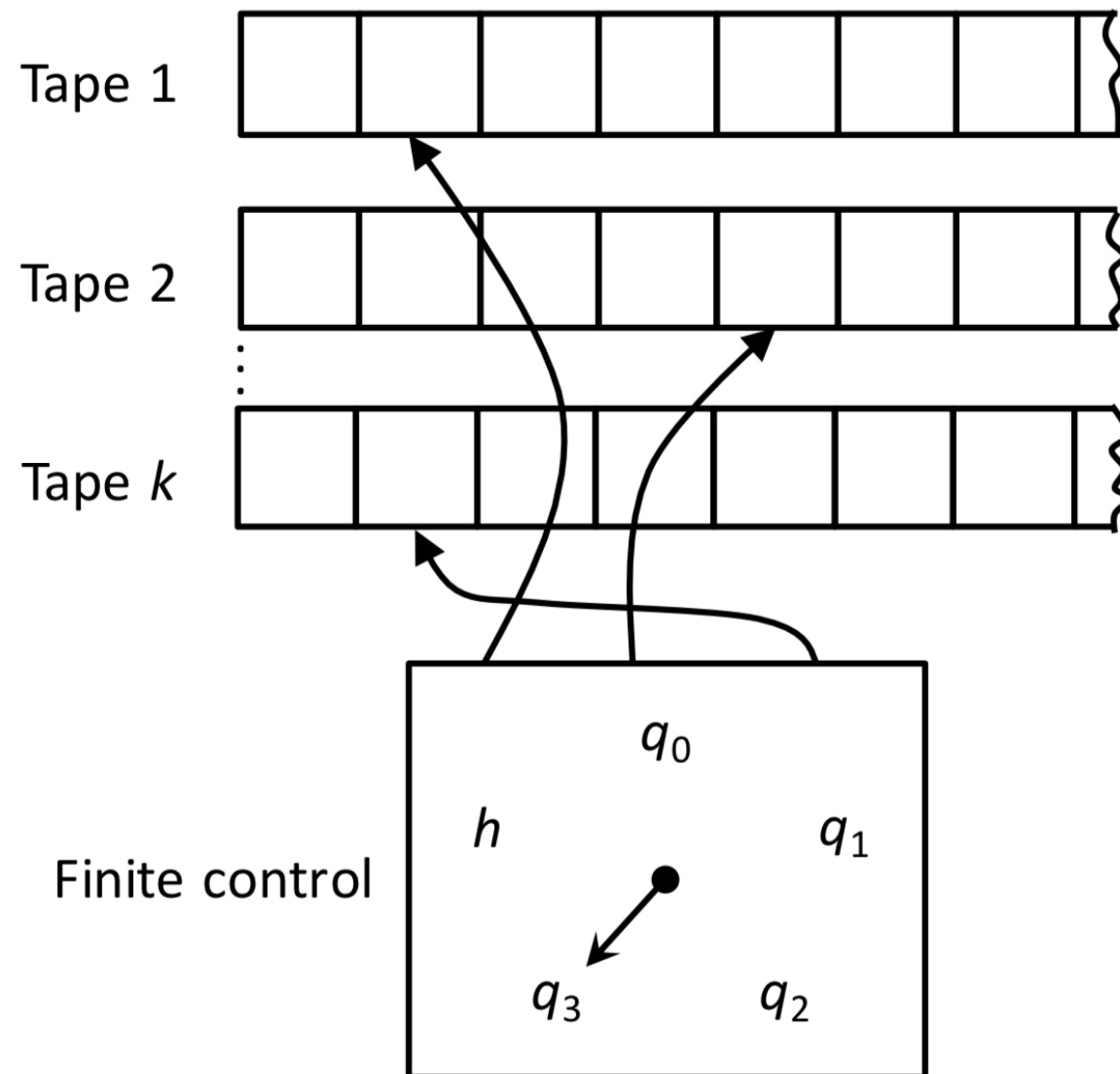
- Many equivalent to define a Turing machine
- Two models are equivalent if they can simulate each other
 - E.g, NFAs and DFAs, CFGs and NPDA, etc
- Invariance to certain changes makes a TM **robust**
- Will discuss two variants:
 - Multitape TM
 - Non-deterministic TM

TM Transition

- Consider a simple change to TM's transition function
- Instead of going left or right, say it can also "stay put"
 - $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{S, L, R\}$
- **Question.** Does such a TM have more power? Why or why not?

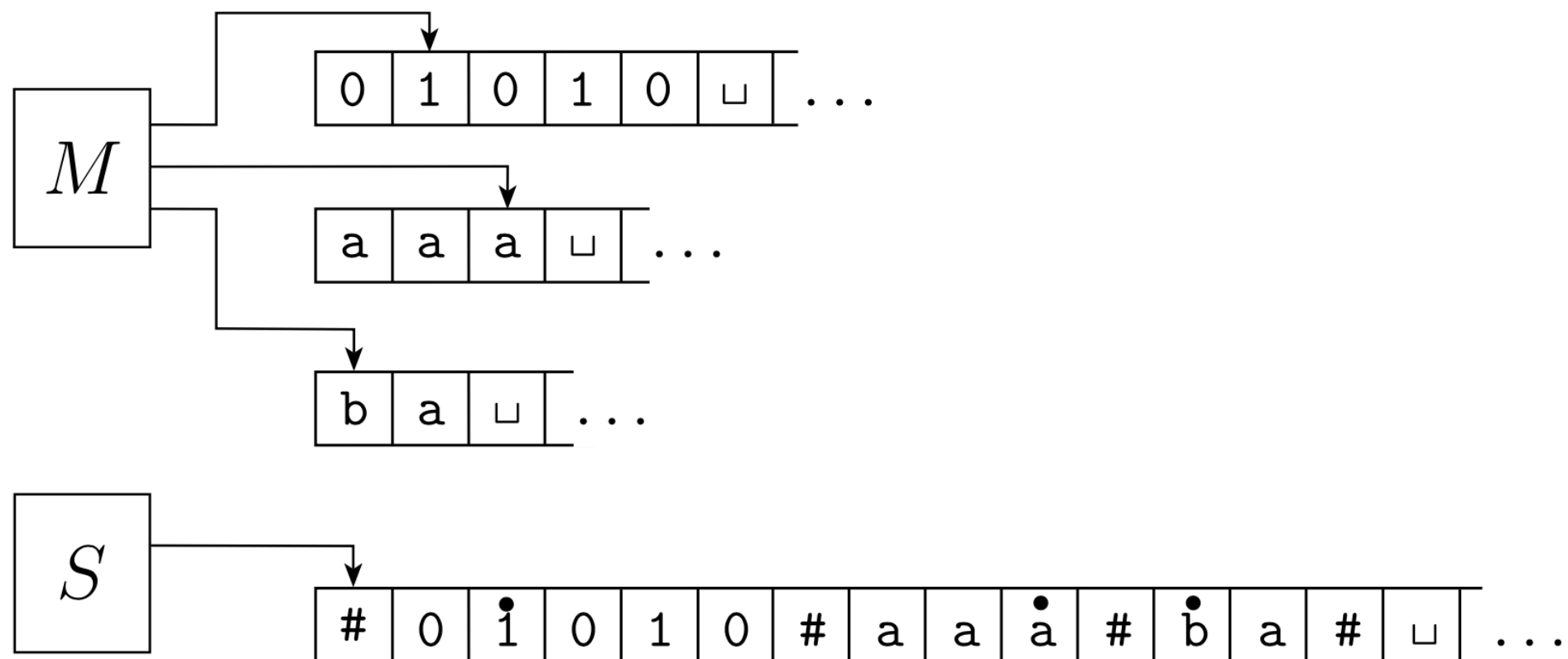
Multitape Turing Machines

- Transition function: $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$



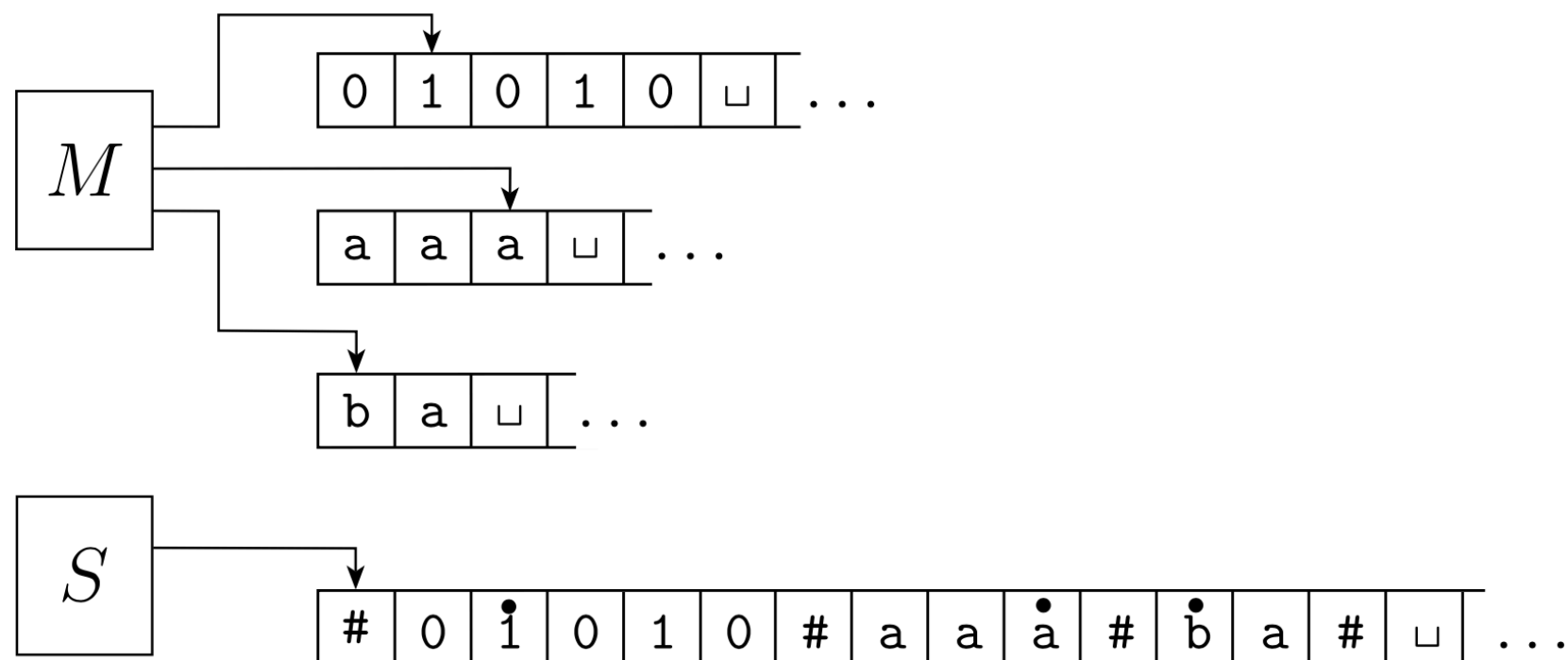
Multitape TM $\iff$ Single Tape TM

- Can a single tape TM simulate a multi-tape TM?
- **Theorem.** Every multi-tape TM has an equivalent single-tape TM.



Multitape TM $\iff$ Single Tape TM

- Can a single tape TM simulate a multi-tape TM?
- **Theorem.** Every multi-tape TM has an equivalent single-tape TM.
- Outline: S uses $\#$ to demarcate single tape into k parts
- Uses special tape symbols $\overset{\circ}{a}$ to indicate head location on each tape



Nondeterministic TM Machine

- At any point in the computation, the TM can "guess" and be in one of several different possibilities
- Transition function:
$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$
- The computation of a NTM is a tree whose branches correspond to different possibilities for the machine
- If some branch leads to an accepts state, the machine accepts
- Intuition for non-determinism: "perfect guessing"
 - When there are several options, magically guess the correct one

Designing NTM

- When designing NTMs, it is often useful to use the **guess-and-check** approach
- Non-deterministically guess a string that can prove $w \in L$
- Deterministically verify that the guess is correct
 - If $w \in L$, there is some guess that works
 - If $w \notin L$, then no guess will work

Designing NTM

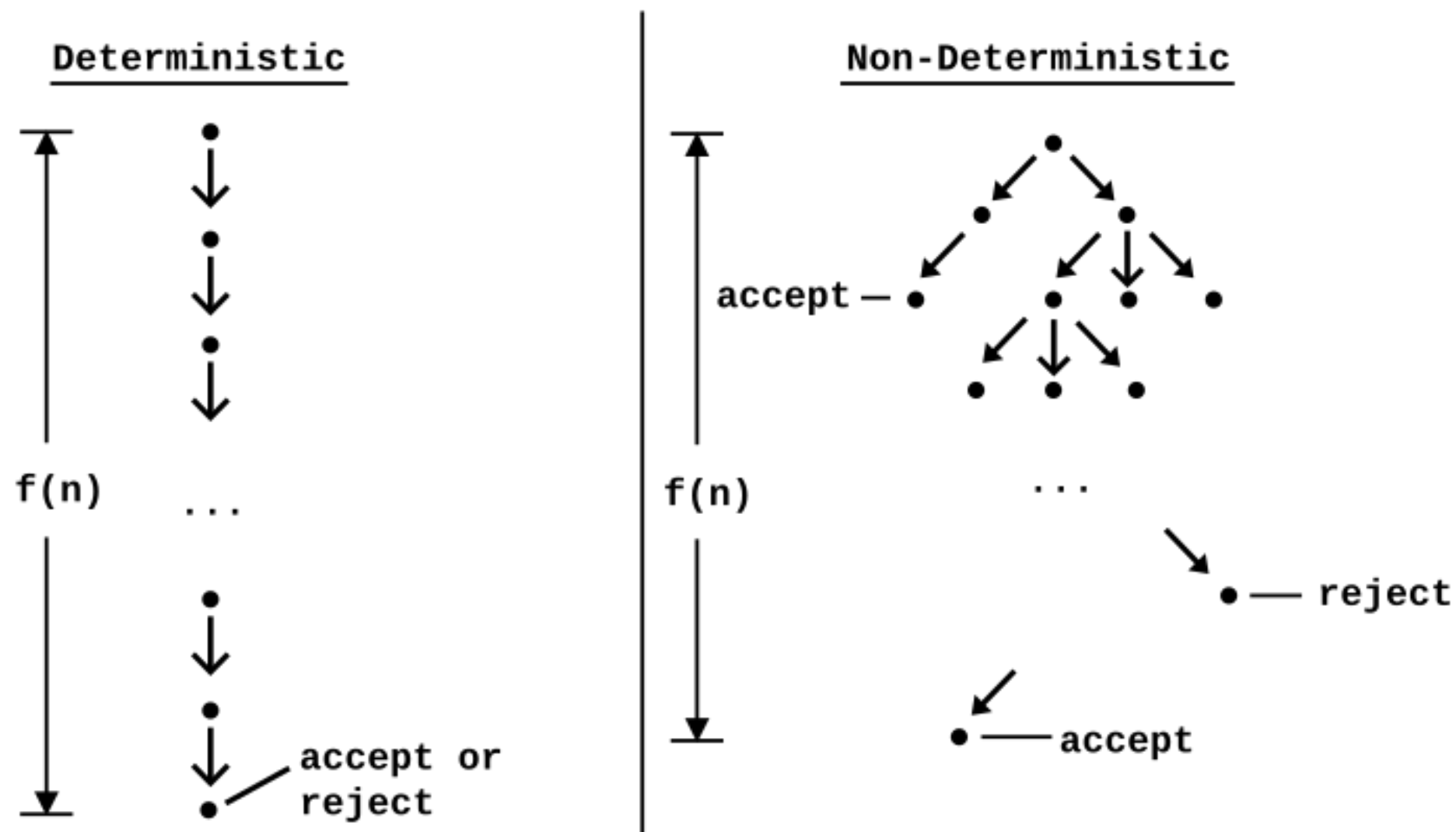
- **Problem.** Design a NTM for the language $L = \{1^n \mid n \text{ is composite}\}$
- A non-deterministic TM for it?
 - Guess a number $1 < m < n$
 - Divide n by m , if it is divisible then accept
 - Else, reject

Nondeterminism

- Non-deterministic finite automata is equivalent to deterministic
- Is non-deterministic push-down automata equivalent to deterministic?
 - What is the intuition behind this?
- Need multiple stacks to keep track of non-deterministic guesses
- Turns out, 2-stack PDA is more powerful than 1-stack PDA
 - In fact, 2-stack PDA $\iff$ Turing machine
 - Intuition behind this?
- **Question.** Is a non-deterministic TM more powerful than a deterministic TM?

NTM $\iff$ DTM

- **Theorem.** Every nondeterministic TM has an equivalent deterministic TM.

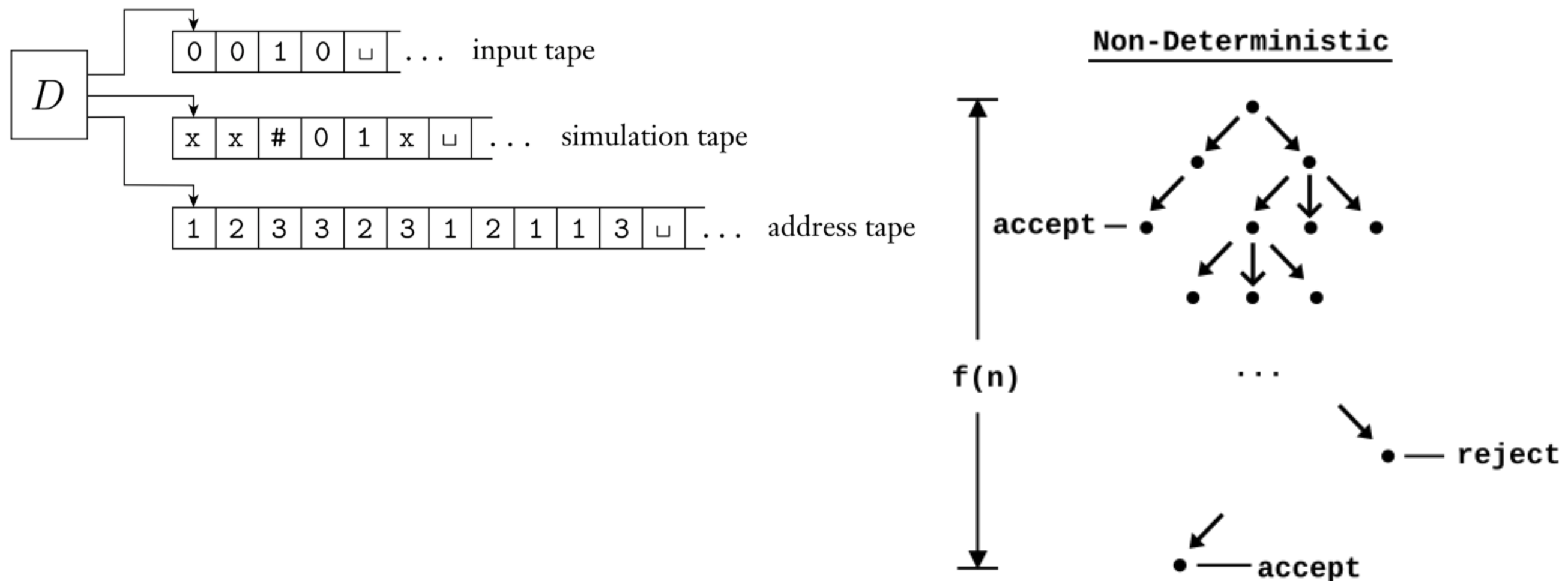


NTM $\iff$ DTM

- **Theorem.** Every nondeterministic TM has an equivalent deterministic TM.
- Intuition. Use a multi-tape machine to simulate NTM
 - Computation of NTM is a tree
 - Each node in the tree is a TM configuration
 - Try all possible branches by traversing the tree
 - Which traversal to do: DFS or BFS?
 - If an accept is ever reached, accept

NTM $\iff$ DTM

- **Proof Idea.** Three tapes: input tape is never altered, simulation tape stores the tape contents of the "current branch" being simulated, address tape keeps track of which node is being traversed in the tree



Main Takeaways

- Alternate characterizations
 - A language is TM recognizable iff some NTM recognizes it
 - A language is TM decidable iff some NTM decides it
- Will revisit NTMs vs DTMs when discussing time complexity
 - P versus NP problem
- Many different ways to define a TM: all are equivalent
 - Makes it a good universal model of computation

Why Study Turing Machines

- Not a good model to think about *fast* computation
- Fast algorithms are a subject of CS 256
- In this class, we are interested in finding out if we can solve a problem at all
- To show a problem is not solvable, we need a model of what it means to solve a problem
 - Church-Turing Thesis:

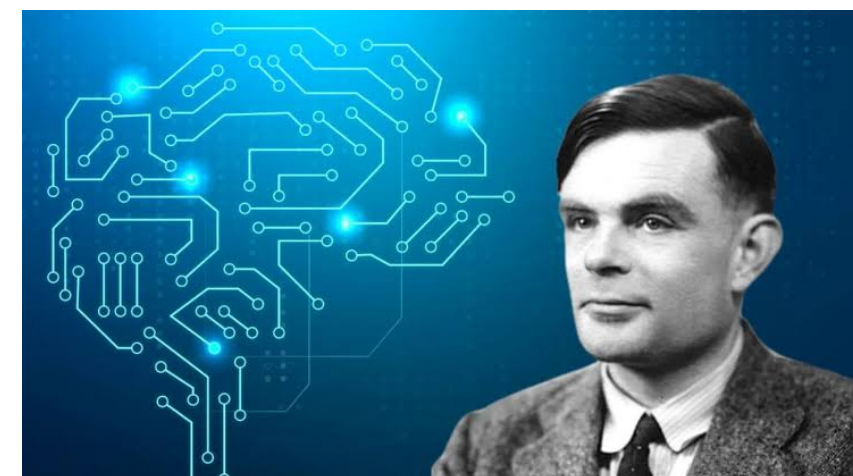
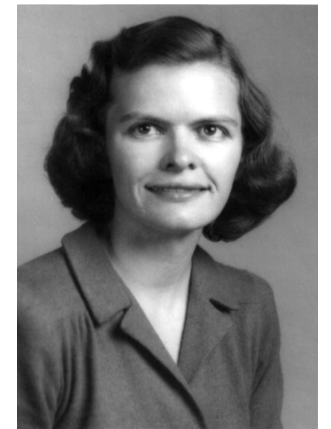
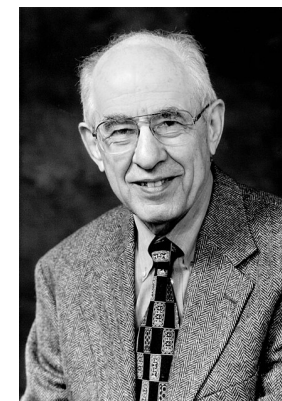
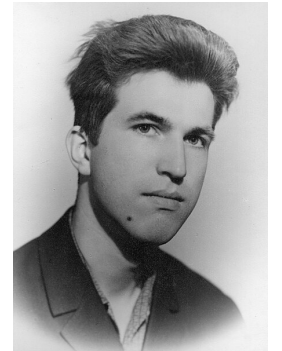
*Intuitive notion
of algorithms*

equals

*Turing machine
algorithms*

Recall: Hilbert's Challenges

- Hilbert's 10th problem [1900]:
 - Given a multivariate polynomial with integer coefficients, is there a **process that determines in a finite number of operations** whether the equation is solvable
 - **No:** Martin Davis, Yuri Matiyasevich, Hilary Putnam and Julia Robinson [1970]
- Hilbert's Entscheidungsproblem (Decision problem) [1928]:
 - **Is there a finite procedure** that determines whether a given mathematical statement is true or false?
 - **No:** Alan Turing [1936]



Church-Turing Thesis Discussion

- A natural law of computation
 - Similar to laws of other physical sciences
- Scott Aaronson: *“whatever it is, the Church-Turing thesis can only be regarded as extremely successful”*
- No candidate computing device (including quantum computers) have posed a serious challenge to it
 - New devices might make things more efficient but do not change the nature of what is fundamentally computable

Towards Unsolvability

- Moving towards finding out what problems are unsolvable
 - That is, not decidable by Turing machines
- These problems are about the behavior of other machines
- Programming language analog:
 - Once you have a PL, we write an interpreter for the language
 - Python interpreter for Python programs
 - Input is any Python program
- Code is data: can be input to programs/TM

PL View: Syntactic vs Semantic Properties

- **Semantic property:** properties that do not depend on the "code/description" of the program but rather depend on the function it computes (about its behavior on inputs)
 - Program accepts some string containing an even # of 1s
- **Syntactic property:** depend on the particular syntax of the program (the description of the machine itself)
 - Program contains an if-else block
- Software verification is all about making sure programs meet certain specifications/requirements
- PL analog for what's coming: **How hard is software verification really?**

Code is Data

- Will soon design TMs that take other TMs as input and try answer semantic properties of the input TM
- To get ready, we will first spend some time showing that when the model of computation is restricted (automaton/CFGs), we can actually answer semantic questions about them
- All these questions will be framed as decision problems

Example Problems

- In general, is it useful to be able to check a program's output on a given input: constantly used in verification
- Acceptance problems: is this string accepted by this machine?
 - $A_{\text{DFA}} = \{ \langle M, w \rangle \mid M \text{ is a DFA and } w \in L(M) \}$
 - Similar variant for NFA and regular expression
 - $A_{\text{CFG}} = \{ \langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G) \}$
 - Similar variant for PDA
 - $A_{\text{TM}} = \{ \langle M, w \rangle \mid T \text{ is a TM and } w \in L(M) \}$

Encodings: Code is Data

- Notice the use of angular brackets to represent the input
 - E.g. $\langle G \rangle$ is the encoding of G if G is a graph
- All these languages have inputs that are machines
- Need a way to encode general objects such as DFAs, CFGs, TMs into strings over a finite alphabet
- The exact encoding scheme is usually unimportant and rarely explicitly discussed
- In the first example we do, we will discuss the encoding of a DFA
- In later examples, we omit these details and assume the TM is given a reasonable encoding

Switch to Algorithms

- As Sipser puts it, we are at a "turning point" in the theory of computation
- Using the Church-Turing thesis, we will assume that TM can implement the algorithms we have discussed in class
 - E.g., a TM can convert a regular expression to an NFA, a PDA to a CFG, etc.
 - The details of how this conversion is handled is not important

Acceptance Problem for DFAs

Theorem. The language

$A_{\text{DFA}} = \{\langle M, w \rangle \mid M \text{ is a DFA and } w \in L(M)\}$ is decidable.

- Assume the DFA $M = (Q, \Sigma, \delta, q_0, F)$ is encoded as a string consisting of the states, followed by alphabet, followed by a list of triples $(q, x, \delta(q, x))$, start state and final states (each separated by #)
- Simulate the DFA M on w (how would this work?)
- Accept if M accepts, otherwise reject

Using Algorithms and Subroutines

Theorem. The following languages are decidable.

$$A_{\text{NFA}} = \{ \langle N, w \rangle \mid N \text{ is a NFA and } w \in L(N) \}$$

$$A_{\text{REG}} = \{ \langle R, w \rangle \mid R \text{ is a regular expression and } w \in L(R) \}$$

What about CFGs?

Theorem. $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$ is decidable.

Question. How do we check if a grammar accepts a given string?

What about CFGs?

Theorem. $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$ is decidable.

Proof. NTM that decides this language:

On input $\langle G, w \rangle$ where G is a CFG:

- Starting from the start state, guess a transition from G
- Keep guessing until a string of non-terminals is reached
- If w is the same as string produced, accept; else reject

What about TMs?

Question. Is $A_{\text{TM}} = \{ \langle M, w \rangle \mid T \text{ is a TM and } w \in L(M) \}$ decidable?

